

[← Go Back](#)

Hardware

Nano 33 IoT

Tutorials

Controlling a LED Through Bluetooth® with Nano 33 IoT

Connecting Two Nano 33 IoT Boards Through I2C

Sending an Email Through IFTTT with Nano 33 IoT

Accessing Accelerometer Data on Nano 33 IoT

Accessing Gyroscope Data on Nano 33 IoT

Nano 33 IoT Alarm with the Arduino Cloud

Connecting Two Nano 33 IoT Boards Through UART

Connecting the Nano 33 IoT to a Wi-Fi Network

Home / Hardware / Nano 33 IoT / **Nano 33 IoT Alarm with the Arduino Cloud**

Nano 33 IoT Alarm with the Arduino Cloud

Learn how to create an alarm with the Nano 33 IoT controlled by the Arduino Cloud

Author • Fabricio Troya

Last revision • 07/17/2024

In this tutorial we will use an Arduino Nano 33 IoT, to set an alarm that will turn on an LED controlled by the Arduino Cloud, using the `RTCZero` library.

Goals

The goals of this project are:

Learn what IoT is.

Learn about RTC (real-time clock).

Use the Arduino RTCZero library.

Learn how to create a new thing in the Arduino Cloud.

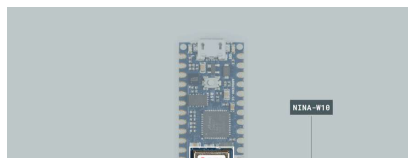
Learn how to set an alarm from the IoT Cloud.

Hardware & Software Needed

This project uses no external sensors or components.

In this tutorial we will use the [Arduino Cloud](#) to program the board.

Internet of Things and Arduino Cloud



ON THIS PAGE

Goals

Hardware & Software Needed

Internet of Things and Arduino Cloud

RTC - Real Time Clock

RTCZero Library

Setting up the Arduino Cloud

Testing It Out

Conclusion

+

Internet of Things, or IoT, is referring to a giant network of connected devices that are collecting and sharing data all over the world. Billion of devices are simultaneously connected to the Internet sharing information with each other and contributing to a vast digital ecosystem.

This vision of a connected world, brings a new computing paradigm where computation happens in a distributed manner, but data is later shared and processed at different centralized pieces of infrastructure. This network of interconnected entities includes lamps, cellphones, smart watches, washing machines, cars, houses, buildings and people amongst other. All of these interconnected entities “talk” to each other using various technologies and protocols and exchange information over the internet. The nature of applications vary from a simple phone-controlled light bulb, to a complex garbage-management system.

Arduino Cloud is an easy-to-use Internet of Things application platform. Arduino Cloud makes the creation of connected objects quick, simple, and secure. You can connect multiple devices and allow them to exchange real-time data. You can also monitor them from anywhere using a simple user interface.

RTC - Real Time Clock

A real-time clock is a clock that keeps track of the current time and can be used to program specific actions at a certain time. The term **real-time clock** is used to avoid confusion with ordinary hardware clocks which are only signals that govern digital electronics, and do not count time in human units. RTC should not be confused with real-time computing.

Most RTCs use a crystal oscillator whose frequency is 32.768 KHz (same frequency used in quartz clocks and watches). Namely this frequency equal to 2^{15} cycles per second and so is a convenient rate to use with simple binary counter circuits. Furthermore, the RTC continues

programmed way. Every time the board is powered, the RTC is reset and starts from standard date. To keep the time and the RTC running it is necessary to keep the device powered.

RTCZero Library

This library allows us to control and use the internal RTC (Real Time Clock) in all the Arduino boards with the **samd** architecture. The library takes care of the initialization of the internal RTC and includes some of the following functions:

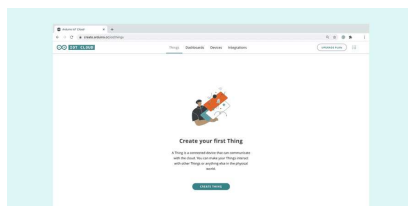
```
1 begin()
2 setTime()
3 setDate()
4 getHours()
5 getMinutes()
6 getSeconds()
7 getYear()
8 getMonth()
9 getDay()
10 setAlarmTime()
11 setAlarmDate()
12 enableAlarm()
13 disableAlarm()
```

If you want to read more about the RTCZero library see [here](#).

Setting up the Arduino Cloud

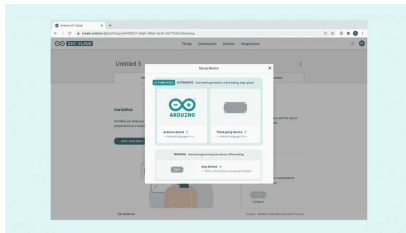
1. Creating the thing

Let's start by opening the [Arduino Cloud](#). Once we are in the Arduino Cloud, we will see the **Things** where we need to click the **Create Thing** button and rename it as **Nano Alarm**.



2. Configuring the device

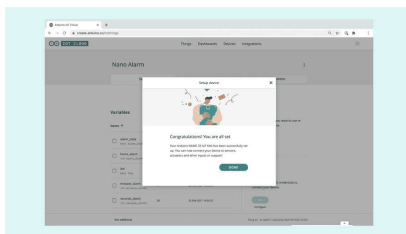
We need to connect the Arduino Nano 33 IoT to the computer and make sure that the IoT Cloud recognizes it by clicking on the **Select Device** button on the device field of the thing. A pop up window will appear, asking you to configure either an Arduino device, or an ESP device. Here, we will choose the **Set up an Arduino device** option.



Setting up a device.

Then, we need to click the **Configure** button, and next we will be asked to name our device. A name is automatically generated, but we can choose any name we want, and click **Next** to continue.

This will initiate a configuration sequence. If nothing goes wrong, we will see a screen below telling us it was successful. Simply press the **Done** button to finish the configuration. If something goes wrong, it could be due to the firmware version (check the note below) or because of the USB cable. Make sure you have an updated version of the Create Plugin on the computer and try to connect the device to the computer with another USB cable.



Setup was successful.

Note: When we configure the board the first time, in order to provide users with the best performance, the Cloud checks if the core

version is not the latest you can upload it following [these instructions](#)

3.Creating variables

Let's begin by clicking on the **Add variable** button. This will open up a new window, where we can choose the **name**, **data type**, **permission setting** and **update setting**.

Next, we need to set up the following variables in the Cloud. All of them should be configured as **On change** following the next specifications:

Name	Data type	Variable permission	Function
alarm_state	boolean	Read & Write	Activate or deactivate the alarm
hours_alarm	int	Read & Write	Set the hours for the alarm
minutes_alarm	int	Read & Write	Set the minutes for the alarm
seconds_alarm	int	Read & Write	Set the seconds for the alarm
led	boolean	Read Only	Indicate if the alarm has been activated or not

Remember, we will also need to make sure that we enter our network credentials. This is done by clicking on the **Configure** button at the **Network** panel in the Setup tab of our thing.

4. Creating the dashboard

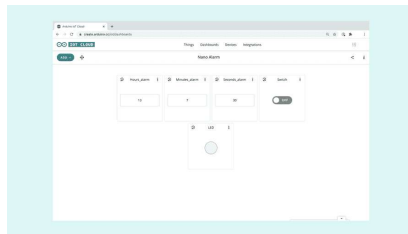
Let's head over to the **Dashboard** tab in the Arduino Cloud, and create a new dashboard by clicking on the **Build Dashboard** button. It is time to rename out "Untitled" dashboard as **Nano Alarm** and start adding the different

one of the widgets, we will need to link them to the variables we created in the previous step, check the table below to know the matchings between widgets and variables.

The dashboard needs five widgets in total, each of them is described in the table below.

Widget	Variable linked
value	hours_alarm
value	minutes_alarm
value	seconds_alarm
switch	alarm_state
led	led

Once we have created all of the widgets, we can arrange them however we want pressing the **arrows** button on the top left, but to keep it clear and simple, it should look similar to the image below.



The dashboard widgets.

5. Building the code

All of the configurations we have done so far, have basically been compiled into a special sketch. This sketch now contains information on what device we are using, a unique ID of our thing, the variables we created and the network information we entered. All we have to do now, is to add some code to the sketch!

Let's go back to our thing overview by navigating to the **Things** tab, then select our thing named "Nano Alarm" and click on the **Sketch** tab. Here, we should see an automatically generated sketch. This sketch is practically a "shell" for the code that we will add next, meaning it has no functionality yet. Since the code we are creating is quite long, we can press the **Open full editor** button that will link us to the Arduino Cloud

Once we are in the Arduino Cloud Editor, we can start modifying the code in order to create an alarm controlled through the IoT Cloud.

After including the `thingProperties` library, we will need to include the `RTCZero` library and create and name an object in this library. In this case, we have called the object `alarm` but you can call it as you want.

```
1 #include "thingProperties.h"
2 #include "RTCZero.h"
3
4 RTCZero alarm;
```

At the beginning of the `setup()`, we need to initialize the Serial Communication at 9600 bauds using a `Serial.begin(9600)`:

```
1 void setup() {
2   // Initialize serial and wait for
3   Serial.begin(9600);
```

Then at the end, after the

`ArduinoCloud.printDebugInfo` function we need to initialize the **RTCZero** library using the `.begin()` function, then we will set the **led** variable as `false`, this will keep the LED off if the alarm hasn't been activated.

```
1 ArduinoCloud.printDebugInfo();
2
3 alarm.begin();
4
5 led = false;
6 }
```

In the `loop()` section, after the `ArduinoCloud.update` function we will use some `Serial.print` functions followed by a `.getHours`, `.getMinutes` and `.getSeconds` functions to keep track and print the current time in the Serial Monitor.

```

1 void loop() {
2   ArduinoCloud.update();
3
4   Serial.print(alarm.getHours());
5   Serial.print(":");
6   Serial.print(alarm.getMinutes());
7   Serial.print(":");
8   Serial.println(alarm.getSeconds());
9 }

```

Now, we will see that after the `loop()` there are some empty functions. These functions have been created automatically from the Cloud and shouldn't be deleted or the sketch won't compile. These functions are created when we add a new variable with **Read & Write** permission, and are called each time we change the value of the variable from the Cloud. So, in order to set a new alarm time each time we change the value from the Cloud, we need to add a `.setAlarmTime()` function inside the following empty functions.

```

1 void onHoursAlarmChange() {
2   alarm.setAlarmTime(hours_alarm,
3 }
4
5 void onMinutesAlarmChange() {
6   alarm.setAlarmTime(hours_alarm,
7 }
8
9 void onSecondsAlarmChange() {
10  alarm.setAlarmTime(hours_alarm,
11 }

```

The `onAlarmStateChange()` function will be called each time we press the switch button from the Cloud. We want to activate the alarm when the switch is on (`true`) and deactivate it when the switch is off (`false`). To do this, we need to add an `if else` statement to check if the switch value is true or false. If the switch value is `true`, we will use the `.setAlarmTime()`, `.enableAlarm()` and `.attachInterrupt()` functions to activate the alarm. Inside the `.attachInterrupt()` function we will add a new function called `alarmMatch` that will control what happens when the alarm

a `.disableAlarm()` function to deactivate the alarm and we will change the led value to `false`.

```
1 void onAlarmStateChange() {
2   if (alarm_state == true) {
3     alarm.setAlarmTime(hours_alarm);
4     alarm.enableAlarm(alarm.MATCH);
5     alarm.attachInterrupt(alarmMatch);
6   }
7   else {
8     alarm.disableAlarm();
9     led = false;
10  }
11 }
```

The last step is to create a new function called `alarmMatch()` that will control what happen when the alarm match. In this case, we will use this function to turn on the LED on the Cloud setting the led value to `true`.

```
1 void alarmMatch() {
2   led = true;
3 }
```

Now the code is complete!

6. Complete code

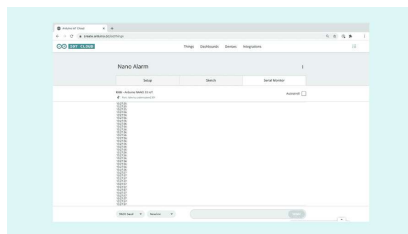
```

1  /*
2   Sketch generated by the Arduino
3   Arduino Cloud Variables descri
4   The following variables are au
5   int hours_alarm;
6   int seconds_alarm;
7   int minutes_alarm;
8   bool alarm_state;
9   bool led;
10  Variables which are marked as l
11  which are called when their va
12  These functions are generated
13  */
14
15  #include "thingProperties.h"
16  #include "RTCZero.h"
17
18  RTCZero alarm;
19
20  void setup() {
21    // Initialize serial and wait
22    Serial.begin(9600);
23
24    // This delay gives the chance
25    delay(1500);
26
27    // Defined in thingProperties.h
28    initProperties();
29  }

```

Testing It Out

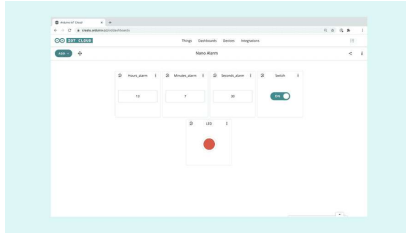
Once we are finished with the coding, we can upload the sketch to the board. When it has successfully uploaded, open the Serial Monitor. In the Serial Monitor, after initializing the communication between the board and the Cloud, we will see the time set as HH:MM:SS.



Printing the time in the Serial Monitor.

We can now go to the **Dashboard** tab in the Arduino Cloud to the **Nano Alarm** dashboard we created before. There we can set time for the alarm and activate the alarm with the switch. If

the switch is on and the alarm match we will see the LED turn on.



Setting an alarm through the dashboard.

Troubleshoot

Sometimes errors occur, if the code is not working there are some common issues we can troubleshoot:

- Missing a bracket or a semicolon.

- Arduino board not connected to the IoT Cloud.

- We haven't opened the Serial Monitor to initialize the program.

- The widgets are not linked to the variables

Conclusion

In this tutorial we have created a basic alarm that turns on a LED through the Arduino Cloud. We learned how to use the Arduino Cloud, how to create a new Thing, create Variables and add widgets to the Dashboard. Lastly, we learned how to use the `RTCZero` library to utilize the RTC peripheral and activate an action at a certain moment.

Suggest changes

The content on docs.arduino.cc is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

